

✓ Data Retrieval From ChEMBL Database

My name is **Dr. Ashfaq Ahmad**, and this tutorial is inspired from the TechOpenCADD project of Volkamer Lab. With some minor modifications, I tried to make it available for the students working with a Biology background. For Proper understanding and learning, you can follow the video tutorial on **Bioinformatics Insights** https://www.youtube.com/channel/UC2Z_WaqTjbvXGGQNpIF9nAg

This notebook is majorly designed for teaching purposes.

For citation and reading, <https://jcheminf.biomedcentral.com/articles/10.1186/s13321-019-0351-x>

Learning and Training Objectives

1. Retrieving compound data and activities from ChEMBL for a protein (receptor) of your choice.

Understanding units

1. IC50 is a standard unit used in biochemical assays for inhibition. It stands for Inhibitory concentration at 50 or Half maximal inhibitory concentration. The lower value indicates higher potency of a drug. You may also see **pIC50**, that is $= -\log_{10}(\text{IC}_{50})$. It makes the opposite of IC50, meaning higher pIC50 means higher potency of a drug.

Let's begin,

✓ Pre-requisite Installation and Imports

```
! pip install chembl_webresource_client  
! pip install rdkit-pypi
```



[Show hidden output](#)

```
import math
from pathlib import Path
from zipfile import ZipFile
from tempfile import TemporaryDirectory

import numpy as np
import pandas as pd
from rdkit.Chem import PandasTools
from chembl_webresource_client.new_client import new_client
from tqdm.auto import tqdm
```

```
HERE = Path(_dh[-1])
DATA = HERE / "data"
```

Creating some API tokens for target

```
targets_api = new_client.target
compounds_api = new_client.molecule
bioactivities_api = new_client.activity
```

```
type(targets_api)
```



```
chembl_webresource_client.query_set.QuerySet
def __init__(model=None, query=None)
```

```
/usr/local/lib/python3.11/dist-packages/chembl_webresource_client/query_set.py
```

In this tutorial we will use **Epidermal Growth Factor (EGFR)** protein, which is a kinase by nature. We will fetch them from Uniprot Accession ID.

```
uniprot_id = "P00533"
```

```
# Get target information from ChEMBL but restrict it to specified values only
targets = targets_api.get(target_components__accession=uniprot_id).only(
    "target_chembl_id", "organism", "pref_name", "target_type"
)
print(f'The type of the targets is "{type(targets)}"')
```



```
The type of the targets is "<class 'chembl_webresource_client.query_set.QuerySet'"
```

```
targets = pd.DataFrame.from_records(targets)
targets
```



	organism	pref_name	target_chembl_id	target_type
0	Homo sapiens	Epidermal growth factor receptor erbB1	CHEMBL203	SINGLE PROTEIN
1	Homo sapiens	Epidermal growth factor receptor erbB1	CHEMBL203	SINGLE PROTEIN
2	Homo sapiens	Epidermal growth factor receptor and ErbB2 (HE...	CHEMBL2111431	PROTEIN FAMILY
3	Homo sapiens	Epidermal growth factor receptor	CHEMBL2363049	PROTEIN FAMILY
4	Homo sapiens	MER intracellular domain/EGFR extracellular do...	CHEMBL3137284	CHIMERIC PROTEIN
5	Homo sapiens	Protein cereblon/Epidermal growth factor receptor	CHEMBL4523680	PROTEIN-PROTEIN INTERACTION
6	Homo sapiens	EGFR/PPP1CA	CHEMBL4523747	PROTEIN-PROTEIN INTERACTION
7	Homo sapiens	VHL/EGFR	CHEMBL4523998	PROTEIN-PROTEIN INTERACTION
8	Homo sapiens	Baculoviral IAP repeat-containing protein 2/Ep...	CHEMBL4802031	PROTEIN-PROTEIN INTERACTION





Next steps:

[Generate code with targets](#)[View recommended plots](#)[New interactive sheet](#)

Below we will select the exact ChEMBL data via index options

```
target = targets.iloc[0]
target
```



	0
organism	Homo sapiens
pref_name	Epidermal growth factor receptor erbB1
target_chembl_id	CHEMBL203
target_type	SINGLE PROTEIN

```
chembl_id = target.target_chembl_id
print(f"The target ChEMBL ID is {chembl_id}")
# NBVAL_CHECK_OUTPUT
```

↗ The target ChEMBL ID is CHEMBL203

Below we are going to fetch some important variable of the data for ourself. You can change them if you want to extract some other column.

```
bioactivities = bioactivities_api.filter(
    target_chembl_id=chembl_id, type="IC50", relation "=", assay_type="B"
).only(
    "activity_id",
    "assay_chembl_id",
    "assay_description",
    "assay_type",
    "molecule_chembl_id",
    "type",
    "standard_units",
    "relation",
    "standard_value",
    "target_chembl_id",
    "target_organism",
)
```

```
print(f"Length and type of bioactivities object: {len(bioactivities)}, {type(bioactivities)}")
```

↗ Length and type of bioactivities object: 11634, <class 'chembl_webresource_client.query_set.QuerySet'>

Each entry in our bioactivity set holds the following information:

```
print(f"Length and type of first element: {len(bioactivities[0])}, {type(bioactivities[0])}")
bioactivities[0]
```

↗ Length and type of first element: 13, <class 'dict'>

```
{'activity_id': 32260,
 'assay_chembl_id': 'CHEMBL674637',
 'assay_description': 'Inhibitory activity towards tyrosine phosphorylation for the epidermal growth factor-receptor kinase',
 'assay_type': 'B',
 'molecule_chembl_id': 'CHEMBL68920',
 'relation': '=',
 'standard_units': 'nM',
 'standard_value': '41.0',
 'target_chembl_id': 'CHEMBL203',
 'target_organism': 'Homo sapiens',
```

```
'type': 'IC50',
'units': 'uM',
'value': '0.041'}
```

Download the dataset

```
#Downloading Data
bioactivities_df = pd.DataFrame.from_dict(bioactivities)
print(f"DataFrame shape: {bioactivities_df.shape}")
bioactivities_df.head()
```

DataFrame shape: (11634, 13)

	activity_id	assay_chembl_id	assay_description	assay_type	molecule_chembl_id	relation	standard_units	standard_value	target_chembl_id
0	32260	CHEMBL674637	Inhibitory activity towards tyrosine phosphory...	B	CHEMBL68920	=	nM	41.0	CHEMBL203
1	32267	CHEMBL674637	Inhibitory activity towards tyrosine phosphory...	B	CHEMBL69960	=	nM	170.0	CHEMBL203
2	32680	CHEMBL677833	In vitro inhibition of Epidermal growth factor...	B	CHEMBL137635	=	nM	9300.0	CHEMBL203
3	32770	CHEMBL674643	Inhibitory concentration of EGF dependent auto...	B	CHEMBL306988	=	nM	500000.0	CHEMBL203
4	32772	CHEMBL674643	Inhibitory concentration of EGF dependent auto...	B	CHEMBL66879	=	nM	3000000.0	CHEMBL203

Next steps:

[Generate code with bioactivities_df](#)

[View recommended plots](#)

[New interactive sheet](#)

It is possible that some of the rows show redundant data, but do not worry we will remove that later

Now we want to know how many different units are present in the data

```
bioactivities_df["units"].unique()
```

```
array(['uM', 'nM', 'pM', 'M', '10^3 uM', '10^1 ug/ml', 'ug ml^-1',  
      '10^-1microM', '10^1 uM', '10^-1 ug/ml', '10^-2 ug/ml', '10^2 uM',  
      '10^-3 ug/ml', '10^-2microM', '/uM', '10^-6g/ml', 'mM', 'umol/L',  
      'nmol/L', '10^-10M', '10^-7M', 'nmol', '10^-8M'], dtype=object)
```

Below we will do some tweaking with units to make them uniform

```
bioactivities_df.drop(["units", "value"], axis=1, inplace=True)  
bioactivities_df.head()
```

	activity_id	assay_chembl_id	assay_description	assay_type	molecule_chembl_id	relation	standard_units	standard_value	target_chembl_id
0	32260	CHEMBL674637	Inhibitory activity towards tyrosine phosphory...	B	CHEMBL68920	=	nM	41.0	CHEMBL203
1	32267	CHEMBL674637	Inhibitory activity towards tyrosine phosphory...	B	CHEMBL69960	=	nM	170.0	CHEMBL203
2	32680	CHEMBL677833	In vitro inhibition of Epidermal growth factor...	B	CHEMBL137635	=	nM	9300.0	CHEMBL203
3	32770	CHEMBL674643	Inhibitory concentration of EGF dependent auto...	B	CHEMBL306988	=	nM	500000.0	CHEMBL203
4	32772	CHEMBL674643	Inhibitory concentration of EGF dependent auto...	B	CHEMBL66879	=	nM	3000000.0	CHEMBL203

Next steps:

[Generate code with bioactivities_df](#)
[View recommended plots](#)
[New interactive sheet](#)

Data Processing and Filtration

Delete entries with missing and duplicate values.

Reset index number and rename columns.

```
bioactivities_df.dtypes
```



0

activity_id	int64
assay_chembl_id	object
assay_description	object
assay_type	object
molecule_chembl_id	object
relation	object
standard_units	object
standard_value	object
target_chembl_id	object
target_organism	object
type	object

```
bioactivities_df = bioactivities_df.astype({"standard_value": "float64"})  
bioactivities_df.dtypes
```



0

activity_id	int64
assay_chembl_id	object
assay_description	object
assay_type	object
molecule_chembl_id	object
relation	object
standard_units	object
standard_value	float64
target_chembl_id	object
target_organism	object
type	object



```
bioactivities_df.dropna(axis=0, how="any", inplace=True)
print(f"DataFrame shape: {bioactivities_df.shape}")
```



```
DataFrame shape: (11633, 11)
```

```
print(f"Units in downloaded data: {bioactivities_df['standard_units'].unique()}")
print(
    f"Number of non-nM entries:\n"
    f"{bioactivities_df[bioactivities_df['standard_units'] != 'nM'].shape[0]}"
)
```



```
Units in downloaded data: ['nM' 'ug.mL-1' '/uM']
Number of non-nM entries:    69
```

```
bioactivities_df = bioactivities_df[bioactivities_df["standard_units"] == "nM"]
print(f"Units after filtering: {bioactivities_df['standard_units'].unique()}")
```



```
Units after filtering: ['nM']
```

```
print(f"DataFrame shape: {bioactivities_df.shape}")
```


↗ DataFrame shape: (11564, 11)

```
bioactivities_df.drop_duplicates("molecule_chembl_id", keep="first", inplace=True)
print(f"DataFrame shape: {bioactivities_df.shape}")
```

↗ DataFrame shape: (7334, 11)

```
bioactivities_df.reset_index(drop=True, inplace=True)
bioactivities_df.head()
```

↗

	activity_id	assay_chembl_id	assay_description	assay_type	molecule_chembl_id	relation	standard_units	standard_value	target_chembl_id
0	32260	CHEMBL674637	Inhibitory activity towards tyrosine phosphory...	B	CHEMBL68920	=	nM	41.0	CHEMBL203
1	32267	CHEMBL674637	Inhibitory activity towards tyrosine phosphory...	B	CHEMBL69960	=	nM	170.0	CHEMBL203
2	32680	CHEMBL677833	In vitro inhibition of Epidermal growth factor...	B	CHEMBL137635	=	nM	9300.0	CHEMBL203
3	32770	CHEMBL674643	Inhibitory concentration of EGF dependent auto...	B	CHEMBL306988	=	nM	500000.0	CHEMBL203
4	32772	CHEMBL674643	Inhibitory concentration of EGF dependent auto...	B	CHEMBL66879	=	nM	3000000.0	CHEMBL203

Next steps:

[Generate code with bioactivities_df](#)

[View recommended plots](#)

[New interactive sheet](#)

```
bioactivities_df.rename(
    columns={"standard_value": "IC50", "standard_units": "units"}, inplace=True
)
bioactivities_df.head()
```



	activity_id	assay_chembl_id	assay_description	assay_type	molecule_chembl_id	relation	units	IC50	target_chembl_id	target_organism
0	32260	CHEMBL674637	Inhibitory activity towards tyrosine phosphory...	B	CHEMBL68920	=	nM	41.0	CHEMBL203	Homo sapiens
1	32267	CHEMBL674637	Inhibitory activity towards tyrosine phosphory...	B	CHEMBL69960	=	nM	170.0	CHEMBL203	Homo sapiens
2	32680	CHEMBL677833	In vitro inhibition of Epidermal growth factor...	B	CHEMBL137635	=	nM	9300.0	CHEMBL203	Homo sapiens
3	32770	CHEMBL674643	Inhibitory concentration of EGF dependent auto...	B	CHEMBL306988	=	nM	500000.0	CHEMBL203	Homo sapiens
4	32772	CHEMBL674643	Inhibitory concentration of EGF dependent auto...	B	CHEMBL66879	=	nM	3000000.0	CHEMBL203	Homo sapiens

Next steps:

[Generate code with bioactivities_df](#)[View recommended plots](#)[New interactive sheet](#)

```
print(f"DataFrame shape: {bioactivities_df.shape}")
```



```
DataFrame shape: (7334, 11)
```

Fetch compound data from ChEMBL

Let's have a look at the compounds from ChEMBL which we have defined so far. bioactivity data for: We are going to fetch compound ChEMBL IDs and structures for the compounds linked to our filtered bioactivity data.

```
compounds_provider = compounds_api.filter(
    molecule_chembl_id__in=list(bioactivities_df["molecule_chembl_id"])
).only("molecule_chembl_id", "molecule_structures")
```

Download Compounds

```
compounds = list(tqdm(compounds_provider))
```

```
compounds_df = pd.DataFrame.from_records(
    compounds,
)
print(f"DataFrame shape: {compounds_df.shape}")
```

↳ DataFrame shape: (7334, 2)

```
compounds_df.head()
```

↳

	molecule_chembl_id	molecule_structures
0	CHEMBL6246	{'canonical_smiles': 'O=c1oc2c(O)c(O)cc3c(=O)o...
1	CHEMBL10	{'canonical_smiles': 'C[S+][([O-])c1ccc(-c2nc(-...
2	CHEMBL6976	{'canonical_smiles': 'COc1cc2c(cc1OC)Nc1ncn(C)...
3	CHEMBL7002	{'canonical_smiles': 'CC1(COc2ccc(CC3SC(=O)NC3...

Next steps:

[Generate code with compounds_df](#)[View recommended plots](#)[New interactive sheet](#)

```
compounds_df.dropna(axis=0, how="any", inplace=True)
print(f"DataFrame shape: {compounds_df.shape}")
```

↳ DataFrame shape: (7327, 2)

```
compounds_df.drop_duplicates("molecule_chembl_id", keep="first", inplace=True)
print(f"DataFrame shape: {compounds_df.shape}")
```

↳ DataFrame shape: (7327, 2)

```
compounds_df.iloc[0].molecule_structures.keys()
```

↳ dict_keys(['canonical_smiles', 'molfile', 'standard_inchi', 'standard_inchi_key'])

```
canonical_smiles = []
```

```
for i, compounds in compounds_df.iterrows():
```

```
try:
    canonical_smiles.append(compounds["molecule_structures"]["canonical_smiles"])
except KeyError:
    canonical_smiles.append(None)

compounds_df["smiles"] = canonical_smiles
compounds_df.drop("molecule_structures", axis=1, inplace=True)
print(f"DataFrame shape: {compounds_df.shape}")
```

⇒ DataFrame shape: (7327, 2)

```
compounds_df.dropna(axis=0, how="any", inplace=True)
print(f"DataFrame shape: {compounds_df.shape}")
```

⇒ DataFrame shape: (7327, 2)

Let's output compound bioactivity data

```
print(f"Bioactivities filtered: {bioactivities_df.shape[0]}")
bioactivities_df.columns
```

⇒ Bioactivities filtered: 7334
Index(['activity_id', 'assay_chembl_id', 'assay_description', 'assay_type',
 'molecule_chembl_id', 'relation', 'units', 'IC50', 'target_chembl_id',
 'target_organism', 'type'],
 dtype='object')

```
print(f"Compounds filtered: {compounds_df.shape[0]}")
compounds_df.columns
```

⇒ Compounds filtered: 7327
Index(['molecule_chembl_id', 'smiles'], dtype='object')

Lets merge both the datasets

Merge to contain, molecule ID, IC50, Units, and canonical smiles

```
# Merge DataFrames
output_df = pd.merge(
    bioactivities_df[["molecule_chembl_id", "IC50", "units"]],
    compounds_df,
    on="molecule_chembl_id",
```

)

Reset row indices

output_df.reset_index(drop=True, inplace=True)

print(f"Dataset with {output_df.shape[0]} entries.")

 Dataset with 7327 entries.

output_df.dtypes



```

0
molecule_chembl_id  object
IC50                 float64
units                object
smiles               object

```

output_df.head(10)



	molecule_chembl_id	IC50	units	smiles	
0	CHEMBL68920	41.0	nM	Cc1cc(C)c(/C=C2\C(=O)Nc3ncnc(Nc4ccc(F)c(Cl)c4)...	
1	CHEMBL69960	170.0	nM	Cc1cc(C(=O)N2CCOCC2)[nH]c1/C=C1\C(=O)Nc2ncnc(N...	
2	CHEMBL137635	9300.0	nM	CN(c1ccccc1)c1ncnc2ccc(N/N=N/Cc3ccccc3)cc12	
3	CHEMBL306988	500000.0	nM	CC(=C(C#N)C#N)c1ccc(NC(=O)CCC(=O)O)cc1	
4	CHEMBL66879	3000000.0	nM	O=C(O)/C=C/c1ccc(O)cc1	
5	CHEMBL77085	96000.0	nM	N#CC(C#N)=Cc1cc(O)ccc1[N+](=O)[O-]	
6	CHEMBL443268	5310.0	nM	Cc1cc(C(=O)NCCN2CCOCC2)[nH]c1/C=C1\C(=O)N(C)c2...	
7	CHEMBL76979	264000.0	nM	COc1cc(/C=C(\C#N)C(=O)O)cc(OC)c1O	
8	CHEMBL76589	125.0	nM	N#CC(C#N)=C(N)/C(C#N)=C/c1ccc(O)cc1	

Next steps:

[Generate code with output_df](#)[View recommended plots](#)[New interactive sheet](#)

Convert IC50 values into pIC50 and put it in an extra column

```
def convert_ic50_to_pic50(IC50_value):
    pIC50_value = 9 - math.log10(IC50_value)
    return pIC50_value

# Apply conversion to each row of the compounds DataFrame
output_df["pIC50"] = output_df.apply(lambda x: convert_ic50_to_pic50(x.IC50), axis=1)
```

```
output_df.head()
```

	molecule_chembl_id	IC50	units	smiles	pIC50
0	CHEMBL68920	41.0	nM	<chem>Cc1cc(C)c(/C=C2\C(=O)Nc3ncnc(Nc4ccc(F)c(Cl)c4)...)</chem>	7.387216
1	CHEMBL69960	170.0	nM	<chem>Cc1cc(C(=O)N2CCOCC2)[nH]c1/C=C1\C(=O)Nc2ncnc(N...</chem>	6.769551
2	CHEMBL137635	9300.0	nM	<chem>CN(c1cccc1)c1ncnc2ccc(N/N=N/Cc3cccn3)cc12</chem>	5.031517
3	CHEMBL306988	500000.0	nM	<chem>CC(=C(C#N)C#N)c1ccc(NC(=O)CCC(=O)O)cc1</chem>	3.301030

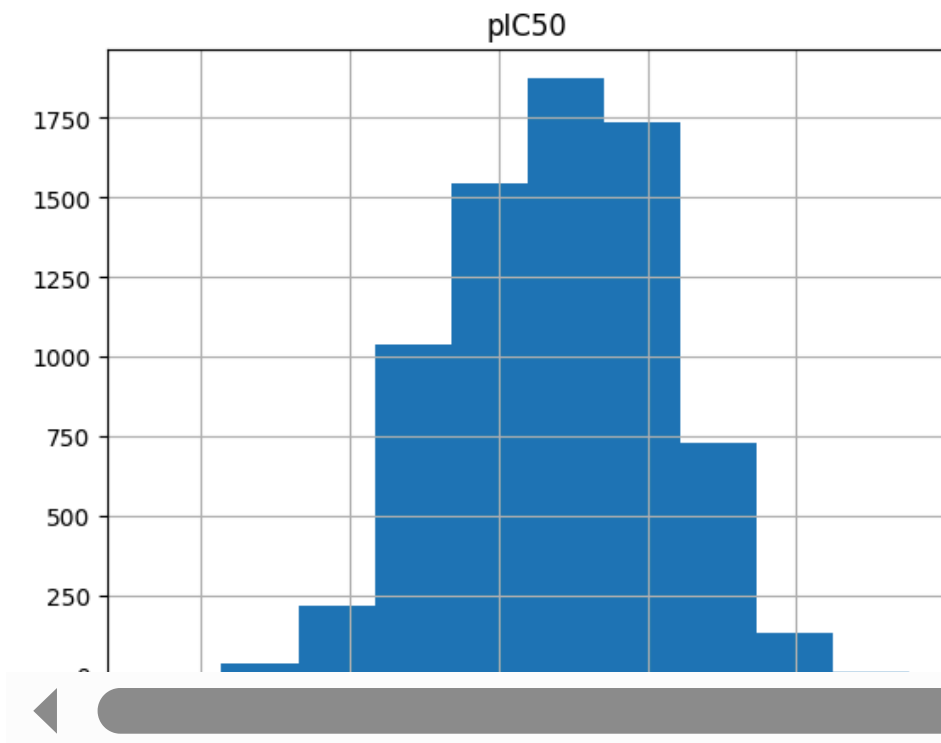
Next steps:

[Generate code with output_df](#)
[View recommended plots](#)
[New interactive sheet](#)

Distribution Based on I_pIC₅₀

```
output_df.hist(column="pIC50")
```

```
array([[<Axes: title={'center': 'pIC50'}>]], dtype=object)
```



In the next steps, we add a column for RDKit molecule objects to our DataFrame and look at the structures of the molecules with the highest pIC50 values.

```
pip install pandas==0.24.2
```

```
Collecting pandas==0.24.2
  Downloading pandas-0.24.2.tar.gz (11.8 MB)
  11.8/11.8 MB 98.0 MB/s eta 0:00:00
error: subprocess-exited-with-error
```

```
× python setup.py egg_info did not run successfully.
  exit code: 1
  ↳ See above for output.
```

```
note: This error originates from a subprocess, and is likely not a problem with pip.
Preparing metadata (setup.py) ... error
```

```
error: metadata-generation-failed
```

```
× Encountered error while generating package metadata.
  ↳ See above for output.
```

note: This is an issue with the package mentioned above, not pip.
hint: See above for details.

```
# Add molecule column
PandasTools.AddMoleculeColumnToFrame(output_df, smilesCol="smiles")
```

Failed to patch pandas - unable to change molecule rendering

```
# Sort molecules by pIC50
output_df.sort_values(by="pIC50", ascending=False, inplace=True)
```

```
# Reset index
output_df.reset_index(drop=True, inplace=True)
```

```
output_df.drop("smiles", axis=1).head(3)
```

	molecule_chembl_id	IC50	units	pIC50	ROMol
0	CHEMBL63786	0.003	nM	11.522879	<rdkit.Chem.rdchem.Mol object at 0x7bac91cc62d0>
1	CHEMBL35820	0.006	nM	11.221849	<rdkit.Chem.rdchem.Mol object at 0x7bac91cc5e00>

```
# Prepare saving the dataset: Drop the ROMol column
output_df = output_df.drop("ROMol", axis=1)
print(f"DataFrame shape: {output_df.shape}")
```

DataFrame shape: (7327, 5)

Save Dataset for Next Session